

Práctica 3 ED | 2º Ingeniería Informática David Rodríguez Aparicio

En esta hemos creado un programa que simula una Agenda de Contactos como puede ser la de un teléfono.

Veremos explicaciones de algunos métodos y sus explicaciones, códigos y algunas imagenes del programa.

Además esta práctica ha sido documentada con Doxygen, creando con Doxyfile una web con toda la estructura de clase.

Para ejecutar el programa, primero hemos hecho "make" en la terminal, desde Practica 3, y a continuación hemos ejecutado agenda.exe, a partir de ahí se nos crea un menú con las distintas opciones del programa, y se carga la agenda de "datos/agendaContactos.txt", que es la que usaremos. (Se inicia automaticamente al inicio del main mediante el constructor de la clase)

Se han añadido la mayoría de características básicas de este tipo de proyectos, pero se pueden extender muchísimo y darle distintas funcionalidades extras.

CONTACTOS.CPP

Este es el archivo cpp, aqui está toda la lógica de la propia clase de Contactos, hay algún que otro método que hemos optado por no usar por lo que están comentado, el resto son bastante intuitivos de su uso y funcionalidad.

También hemos hecho un doble constructor, uno que es para insertarContacto, que te va preguntando los datos necesarios y otro que usamos al crear los contactos al traerlos de un documento.

```
/**
 * @file contactos.cpp
 * @brief Implementación de los métodos de la clase Contacto.
 */

#include "contactos.h"

#include <iostream>
// #include <cassert>
#include <iosfwd>
#include <sstream>

#include <string>
#include <set>
#include <map>

using namespace std;

/* ----- CONSTRUCTOR ----- */
```

```
Contacto::Contacto(string nombreContacto){

    this->nombre = nombreContacto;

    this->preguntarConstructor();

}

/* SOLO SE USA AL USAR CARGAR ARCHIVO */
Contacto::Contacto(){
    this->nombre = "";
}

/* ----- MÉTODOS ----- */

/* Hacen falta los metodos tiene, set no crea duplicados ????????? */

bool Contacto::añadirTelefono(const string& numTelefono){
    return this->telefonos.insert(numTelefono).second;
}

bool Contacto::borrarTelefono(const string& numTelefono){
    return this->telefonos.erase(numTelefono) > 0;
}

/* bool Contacto::tieneEsteTelefono(const string& numTelefono) const{
    return ( this->telefonos.find(numTelefono) != this->telefonos.end() );
} */

bool Contacto::añadirCorreo(const string& nuevoCorreo){
    return this->correos.insert(nuevoCorreo).second;
}

bool Contacto::borrarCorreo(const string& correoBorrar){
    return this->correos.erase(correoBorrar) > 0;
}

/* bool Contacto::tieneEsteCorreo(const string& correoBuscar) const{
    return ( this->correos.find(correoBuscar) != this->correos.end() );
} */

bool Contacto::añadirEtiqueta(const string& nuevaEtiqueta){
    return this->etiquetas.insert(nuevaEtiqueta).second;
}
```

```
bool Contacto::borrarEtiqueta(const string& etiquetaBorrar){
    return this->etiquetas.erase(etiquetaBorrar) > 0;
}

/* bool Contacto::tieneEsteEtiqueta(const string& etiquetaBuscar) const{
    return ( this->etiquetas.find(etiquetaBuscar) != this->etiquetas.end() );
} */

void Contacto::preguntarConstructor(){

    int cantidadTelefonos,cantidadCorreos,cantidadEtiquetas;

    string añadirTelefono,añadirCorreo,añadirEtiquetas;

    cout << "\n Cuantos telefonos tiene? ";
    cin >> cantidadTelefonos;

    cout << "\n Escribe el numero de los telefonos:";
    for(int i = 0; i<cantidadTelefonos; i++){

        cout << "\n" + to_string(i) + ": ";
        cin >> añadirTelefono;

        this->añadirTelefono(añadirTelefono);
    }

    cout << "\n Cuantos correos tiene?";
    cin >> cantidadCorreos;

    cout << "\n Escribe el nombre de los correos:";
    for(int i = 0; i<cantidadCorreos; i++){

        cout << "\n" + to_string(i) + ": ";
        cin >> añadirCorreo;

        this->añadirCorreo(añadirCorreo);
    }

    cout << "\n Cuantas etiquetas tiene?";
    cin >> cantidadEtiquetas;

    cout << "\n Escribe las etiquetas: ";
    for(int i = 0; i<cantidadEtiquetas; i++){

        cout << "\n" + to_string(i) + ": ";
        cin >> añadirEtiquetas;

        this->añadirEtiqueta(añadirEtiquetas);
    }
}
```

```
    }

}

string Contacto::toString() const {
    /* los foreach usan ya cons_iterator/begin/end por defecto ?; */
    string texto;

    texto = " Contacto: " + nombre;

    texto += "\n Telefonos: ";
    if (telefonos.empty()) {
        texto += "No tiene telefonos";
    } else {
        for (const auto& telefono : telefonos) {
            texto += telefono + " ";
        }
    }

    texto += "\n Correos: ";
    if (correos.empty()) {
        texto += "No tiene correos";
    } else {
        for (const auto& correo : correos) {
            texto += correo + " ";
        }
    }

    texto += "\n Etiquetas: ";
    if (etiquetas.empty()) {
        texto += "No tiene etiquetas";
    } else {
        for (const auto& etiqueta : etiquetas) {
            texto += etiqueta + " ";
        }
    }

    return texto;
}

/* ----- SETTERS / GETTERS ----- */

const string& Contacto::getNombre() const{
    return this->nombre;
}

const set<string>& Contacto::getTelefonos() const{
    return this->telefonos;
}

const set<string>& Contacto::getCorreos() const{
    return this->correos;
}
```

```

}

const set<string>& Contacto::getEtiquetas() const{
    return this->etiquetas;
}

void Contacto::setNombre( string nuevoNombre){
    this->nombre = nuevoNombre;
}

```

AGENDACONTACTOS.CPP

Aquí esta toda la gestión de la Agenda con sus distintas funcionalidades y búsquedas en base a ciertos criterios.

```

/**
 * @file AgendaContactos.cpp
 * @brief Implementación de los métodos de AgendaContactos.
 */

#include "agendacontactos.h"

#include <fstream>
#include <sstream>
#include <iostream>

#include <string>

using namespace std;

/* ----- CONSTRUCTOR ----- */

AgendaContactos::AgendaContactos(const string& nombre_fichero) {
    cargar(nombre_fichero);
}

/* ----- MÉTODOS ----- */

bool AgendaContactos::insertarContacto(const Contacto& nuevoContacto) {

    pair<map<string, Contacto>::iterator, bool> haInsertado = this-
>listaContactos.insert({ nuevoContacto.getNombre(), nuevoContacto });

    const set<string>& etiquetasContacto = nuevoContacto.getEtiquetas();

    for (const string& etiqueta : etiquetasContacto) {

```

```
        this->etiquetas.insert({ etiqueta, nuevoContacto.getNombre() });
    }

    return haInsertado.second;
}

bool AgendaContactos::eliminarContacto(const string& borrarContacto){

    /* Comprobacion de que existe. */

    if ( !this->existeContacto(borrarContacto) ) {
        return false;
    }

    /* Buscamos etiquetas y borramos */
    for (auto i = etiquetas.begin(); i != etiquetas.end(); ) {
        if (i->second == borrarContacto) {
            i = etiquetas.erase(i);
        } else {
            ++i;
        }
    }

    /* Borramos el contacto */
    listaContactos.erase(borrarContacto);

    return true;
}

bool AgendaContactos::existeContacto(const string& nombre) const{
    return ( listaContactos.find(nombre) != listaContactos.end() );
}

const Contacto* AgendaContactos::buscarContacto(const string& nombre) const{
    auto contactoMap = listaContactos.find(nombre);

    if (contactoMap == listaContactos.end()) {
        return nullptr;
    }

    return &(contactoMap->second);
}

string AgendaContactos::contactosPorEtiqueta(const std::string& etiqueta) const{

    string resultado = "\n Contactos con la etiqueta '" + etiqueta + "':\n";

    auto rango = etiquetas.equal_range(etiqueta);

    /* Significaria que las dos partes del multimap están vacías. */
    if (rango.first == rango.second) {
```

```
        return "No hay contactos con la etiqueta '" + etiqueta + "'.";
    }

    for (auto it = rango.first; it != rango.second; ++it) {
        resultado += " - " + it->second + "\n";
    }

    return resultado;
}

string AgendaContactos::toString() const{
    if (listaContactos.empty()) {
        return "Agenda vacia.";
    }

    string resultado = "\n AGENDA DE CONTACTOS \n";

    for (const auto& par : listaContactos) {
        resultado += "\n-----\n";
        resultado += par.second.toString();
    }

    return resultado;
}

bool AgendaContactos::modificarContacto(const string& nombre) {

    auto it = listaContactos.find(nombre);
    if (it == listaContactos.end()) return false;

    Contacto& contactoModificar = it->second;

    int operacion = -1;

    while (operacion != 0) {
        cout << "\n\n--- MODIFICAR CONTACTO ---\n";
        cout << contactoModificar.toString() << "\n";
        cout << "1. Añadir telefono\n";
        cout << "2. Borrar telefono\n";
        cout << "3. Añadir correo\n";
        cout << "4. Borrar correo\n";
        cout << "5. Añadir etiqueta\n";
        cout << "6. Borrar etiqueta\n";
        cout << "0. Volver\n";
        cout << "Opcion: ";

        cin >> operacion;
        cin.ignore(10000, '\n');

        string dato;

        switch (operacion) {
            case 1:
                cout << "Telefono a añadir: ";
```

```
        cin >> dato;
        cout << (contactoModificar.añadirTelefono(dato) ? "Añadido\n" :
"Ya existia\n");
        break;

    case 2:
        cout << "Telefono a borrar: ";
        cin >> dato;
        cout << (contactoModificar.borrarTelefono(dato) ? "Borrado\n" :
"No existia\n");
        break;

    case 3:
        cout << "Correo a añadir: ";
        cin >> dato;
        cout << (contactoModificar.añadirCorreo(dato) ? "Añadido\n" : "Ya
existia\n");
        break;

    case 4:
        cout << "Correo a borrar: ";
        cin >> dato;
        cout << (contactoModificar.borrarCorreo(dato) ? "Borrado\n" : "No
existia\n");
        break;

    case 5:
        cout << "Etiqueta a añadir: ";
        cin >> dato;

        if (contactoModificar.añadirEtiqueta(dato)) {
            etiquetas.insert({dato, nombre});
            cout << "Añadida\n";
        } else {
            cout << "Ya existia\n";
        }
        break;

    case 6:
        cout << "Etiqueta a borrar: ";
        cin >> dato;

        if (contactoModificar.borrarEtiqueta(dato)) {
            auto rango = etiquetas.equal_range(dato);
            for (auto mit = rango.first; mit != rango.second; ) {
                if (mit->second == nombre) mit = etiquetas.erase(mit);
                else ++mit;
            }
            cout << "Borrada\n";
        } else {
            cout << "No existia\n";
        }
        break;
```



```
        case 0:
            break;

        default:
            cout << "Opcion no valida\n";
    }
}

return true;
}

bool AgendaContactos::cargar(const string& nombre_fichero){

    ifstream fichero(nombre_fichero);
    if (!fichero) return false;

    // Si hiciese falta limpiar la Agenda
    // listaContactos.clear();
    // etiquetas.clear();

    string linea;

    while (getline(fichero, linea)) {

        string nombre, telefonos, correos, tags;
        stringstream ss(linea);

        getline(ss, nombre, '|');
        getline(ss, telefonos, '|');
        getline(ss, correos, '|');
        getline(ss, tags, '|');

        Contacto nuevoContacto = Contacto();

        nuevoContacto.setNombre(nombre);

        string dato;

        stringstream ssTel(telefonos);
        while ( getline(ssTel, dato, ',') ) {
            if (!dato.empty()) nuevoContacto.añadirTelefono(dato);
        }

        stringstream ssCor(correos);
        while (getline(ssCor, dato, ',')) {
            if (!dato.empty()) nuevoContacto.añadirCorreo(dato);
        }

        stringstream ssTag(tags);
        while (getline(ssTag, dato, ',')) {
            if (!dato.empty()) nuevoContacto.añadirEtiqueta(dato);
        }
    }
}
```

```
    }

    if( !this->insertarContacto(nuevoContacto) ){
        return false;
    }
}

return true;
}

bool AgendaContactos::guardar(const string& nombre_fichero ) const{

    ofstream fichero(nombre_fichero);
    if (!fichero) return false;

    for (auto it = listaContactos.begin(); it != listaContactos.end(); ++it) {

        const Contacto& c = it->second;

        // Nombre
        fichero << c.getNombre() << "|";

        // Telefonos
        const set<string>& telefonos = c.getTelefonos();
        for (auto itTel = telefonos.begin(); itTel != telefonos.end(); ++itTel) {
            fichero << *itTel;
            auto sig = itTel;
            ++sig;
            if (sig != telefonos.end()) fichero << ",";
        }
        fichero << "|";

        // Correos
        const set<string>& correos = c.getCorreos();
        for (auto itCor = correos.begin(); itCor != correos.end(); ++itCor) {
            fichero << *itCor;
            auto sig = itCor;
            ++sig;
            if (sig != correos.end()) fichero << ",";
        }
        fichero << "|";

        // Etiquetas
        const set<string>& etiquetasContacto = c.getEtiquetas();
        for (auto itEt = etiquetasContacto.begin(); itEt !=
etiquetasContacto.end(); ++itEt) {
            fichero << *itEt;
            auto sig = itEt;
            ++sig;
            if (sig != etiquetasContacto.end()) fichero << ",";
        }

        // Salto de línea salvo último (opcional)
        fichero << "\n";
    }
}
```

```

    }

    return true;
}

/* ----- SETTERS / GETTERS ----- */

int AgendaContactos::getNumeroContactos() const{
    return this->listaContactos.size();
}

string AgendaContactos::getEtiquetas() const{
    string resultado = "\n ETIQUETAS: ";

    for (const auto& Etiqueta : this->etiquetas) {
        resultado += "\n - " + Etiqueta.first + " - " + Etiqueta.second;
    }

    return resultado;
}

```

MAIN.CPP

Y por último el main, que no tiene mucho misterio, es como una vista para la gestión de la Agenda y los mensajes por consola.

```

#include <iostream>
#include "AgendaContactos.h"

using namespace std;

void mostrarMenu() {
    cout << "\n\n===== AGENDA DE CONTACTOS =====\n";
    cout << "1. Mostrar agenda completa\n";
    cout << "2. Insertar nuevo contacto\n";
    cout << "3. Eliminar contacto\n";
    cout << "4. Buscar contacto por nombre\n";
    cout << "5. Comprobar si existe contacto\n";
    cout << "6. Buscar contactos por etiqueta\n";
    cout << "7. Guardar Archivo\n";
    cout << "8. Modificar Contacto\n";
    cout << "0. Salir\n";
    cout << "Seleccione una opcion: ";
}

```

```
}

int leerOpcion() {
    int opcion;
    cin >> opcion;

    while (cin.fail()) {
        cin.clear();
        cin.ignore(10000, '\n');
        cout << "Entrada no valida. Introduce un numero: ";
        cin >> opcion;
    }

    cin.ignore(10000, '\n');
    return opcion;
}

int main(){

    AgendaContactos agenda("./datos/agendaContactos.txt");

    cout << "Agenda Actual: \n";
    cout << agenda.toString();

    int opcion = -1;

    do{

        mostrarMenu();

        opcion = leerOpcion();

        switch(opcion){

            case 1:{
                cout << agenda.toString();
                break;
            }

            case 2:{
                string nombre;

                cout << "Nombre del contacto: ";
                getline(cin, nombre);

                Contacto nuevo(nombre);

                if (agenda.insertarContacto(nuevo))
                    cout << "Contacto insertado correctamente.\n";
                else
                    cout << "El contacto ya existe.\n";

                break;
            }
        }
    }
```

```
case 3:{
    string nombre;
    cout << "Nombre del contacto a eliminar: ";
    cin >> nombre;

    if (agenda.eliminarContacto(nombre))
        cout << "Contacto eliminado.\n";
    else
        cout << "Contacto no encontrado.\n";

    break;
}

case 4:{
    string nombre;
    cout << "Nombre del contacto: ";
    cin >> nombre;

    const Contacto* c = agenda.buscarContacto(nombre);
    if (c != nullptr)
        cout << c->toString() << endl;
    else
        cout << "Contacto no encontrado.\n";

    break;
}

case 5:{
    string nombre;
    cout << "Nombre del contacto: ";
    cin >> nombre;

    if (agenda.existeContacto(nombre))
        cout << "El contacto existe.\n";
    else
        cout << "El contacto NO existe.\n";

    break;
}

case 6:{
    string etiqueta;
    cout << "Etiqueta: ";
    cin >> etiqueta;

    cout << agenda.contactosPorEtiqueta(etiqueta) << endl;
    break;
}

case 7:{
```

```
        if(agenda.guardar("./datos/agendaContactos.txt")){
            cout << "\n Guardado Correctamente.";
        }else{
            cout << "\n Fallo al guardar.";
        }
        break;
    }

    case 8:{

        string nombre;
        cout << "Nombre del contacto: ";
        cin >> nombre;

        if (!agenda.modificarContacto(nombre)) cout << "Contacto no
encontrado.\n";

        break;
    }

    case 0:{
        cout << "Saliendo de la agenda...\n";
        break;
    }

    default: {
        cout << "Opcion no valida.\n";
    }
}

} while (opcion != 0);

return 0;
}
```

IMAGENES DEL PROGRAMA

```
PS C:\Users\Daviton\Desktop\Ingenieria-Informatica-2\ED\Practica 3> make
g++ -std=c++11 -Iinclude -Wall -Wextra -c src/main.cpp -o build/main.o
g++ -std=c++11 -Iinclude -Wall -Wextra -c src/AgendaContactos.cpp -o build/AgendaContactos.o
g++ -std=c++11 -Iinclude -Wall -Wextra -c src/contactos.cpp -o build/contactos.o
g++ build/main.o build/AgendaContactos.o build/contactos.o -o agendaContactos
PS C:\Users\Daviton\Desktop\Ingenieria-Informatica-2\ED\Practica 3> .\agendaContactos.exe
Agenda Actual:
```

AGENDA DE CONTACTOS

```
-----
Contacto: David
Telefonos: 654688355
Correos: david@gmail.com
Etiquetas: amigo
-----
Contacto: Jose
Telefonos: 000000000
Correos: jose@gmail.com jose@hotmail.com
Etiquetas: universidad
-----
Contacto: Julio
Telefonos: 123456789 987654321
Correos: julioc@gmail.com
Etiquetas: amigo
-----
Contacto: Raul
Telefonos: 123123333 123123444
Correos: raulito@gmail.com
Etiquetas: amigo universidad
```

===== AGENDA DE CONTACTOS =====

1. Mostrar agenda completa
 2. Insertar nuevo contacto
 3. Eliminar contacto
 4. Buscar contacto por nombre
 5. Comprobar si existe contacto
 6. Buscar contactos por etiqueta
 7. Guardar Archivo
 8. Modificar Contacto
 0. Salir
- Seleccione una opcion: █

Etiquetas: amigo universidad

===== AGENDA DE CONTACTOS =====

1. Mostrar agenda completa
2. Insertar nuevo contacto
3. Eliminar contacto
4. Buscar contacto por nombre
5. Comprobar si existe contacto
6. Buscar contactos por etiqueta
7. Guardar Archivo
8. Modificar Contacto
0. Salir

Seleccione una opcion: 4

Nombre del contacto: David

Contacto: David

Telefonos: 654688355

Correos: david@gmail.com

Etiquetas: amigo

===== AGENDA DE CONTACTOS =====

1. Mostrar agenda completa
2. Insertar nuevo contacto
3. Eliminar contacto
4. Buscar contacto por nombre
5. Comprobar si existe contacto
6. Buscar contactos por etiqueta
7. Guardar Archivo
8. Modificar Contacto
0. Salir

Seleccione una opcion: 8

Nombre del contacto: David

--- MODIFICAR CONTACTO ---

Contacto: David

Telefonos: 654688355

Correos: david@gmail.com

Etiquetas: amigo

1. Añadir telefono
2. Borrar telefono
3. Añadir correo
4. Borrar correo
5. Añadir etiqueta
6. Borrar etiqueta
0. Volver

Opcion: 2

Telefono a borrar: 654688355

Borrado

--- MODIFICAR CONTACTO ---

Contacto: David

Telefonos: No tiene telefonos

Correos: david@gmail.com

Etiquetas: amigo

1. Añadir telefono
2. Borrar telefono
3. Añadir correo
4. Borrar correo
5. Añadir etiqueta
6. Borrar etiqueta
0. Volver

Opcion: █


```
--- MODIFICAR CONTACTO ---
Contacto: David
Telefonos: No tiene telefonos
Correos: david@gmail.com
Etiquetas: amigo
1. Añadir telefono
2. Borrar telefono
3. Añadir correo
4. Borrar correo
5. Añadir etiqueta
6. Borrar etiqueta
0. Volver
Opcion: 0

===== AGENDA DE CONTACTOS =====
1. Mostrar agenda completa
2. Insertar nuevo contacto
3. Eliminar contacto
4. Buscar contacto por nombre
5. Comprobar si existe contacto
6. Buscar contactos por etiqueta
7. Guardar Archivo
8. Modificar Contacto
0. Salir
Seleccione una opcion: 7

Guardado Correctamente.

===== AGENDA DE CONTACTOS =====
1. Mostrar agenda completa
2. Insertar nuevo contacto
3. Eliminar contacto
4. Buscar contacto por nombre
5. Comprobar si existe contacto
6. Buscar contactos por etiqueta
7. Guardar Archivo
8. Modificar Contacto
0. Salir
Seleccione una opcion: █
```

```
You, hace 19 segundos | 1 author (You)
1 David | david@gmail.com | amigo
2 Jose | 000000000 | jose@gmail.com, jose@hotmail.com | universidad
3 Julio | 123456789, 987654321 | juliio@gmaail.com | amigo
4 Raul | 123123333, 123123444 | raulito@gmail.com | amigo, universidad
5 |
```